

Tuning Collective Patterns to Alleviate Congestion in Shared AI Clusters

Eashan Gupta
UIUC

Yongzhou Chen
Meta

Apoorve Mohan
IBM Research

Pavlos Maniotis
IBM Research

Abdullah Kayi
IBM Research

Radhika Mittal
UIUC

Abstract

Distributed AI training involves recurring rounds of data exchange between multiple pairs of GPU nodes. Slowdown in even one flow due to congestion can cause the entire communication round to slowdown. Current approaches for evading congestion in AI clusters assume global control over the entire workload (e.g. coordinating the schedule of all jobs) or assume infrastructural support (e.g. adaptive routing in switches). They are thus ill-suited in a shared cloud setting where AI jobs belonging to one user can face external congestion from other users’ jobs or background traffic beyond its own control. In this paper, we build a system, REACT, that tunes the recurring *pattern* of data exchange between GPU nodes (known as communication collectives) in response to congestion. REACT works at the application (communication library) layer, where it detects congestion at runtime using readily available flow stats, and tunes the collective pattern to alleviate congestion – changing the set of incident flows while retaining the semantics of information exchange (e.g. selecting which node aggregates data in an AllReduce tree). REACT requires no explicit support from the underlying network infrastructure and can be unilaterally deployed by individual users in a shared cloud setting. We prototype *REACT* as a shim layer over NCCL, and evaluate it on a shared academic GPU cluster – enabling *REACT* improves communication performance (algorithm bandwidth) by 13%–38% under network congestion. Our simulations across a range of congestion scenarios further reveal up to 75% performance improvement, highlighting the effectiveness of our approach.

1 Introduction

Scaling machine learning (ML) and AI training jobs to increasingly larger amounts of data and model sizes requires distributing them across multiple nodes. A typical distributed training job runs as a series of recurring *epochs*, where each epoch is composed of a computation phase (where each node performs a local computation), followed by a communication phase (where relevant data, e.g., gradients and parameters, are exchanged among the nodes). The communication phase involves multiple flows (data exchange between multiple pairs of nodes). Due to synchronization barriers, slow down in even one flow (e.g. due to localized congestion) can slow down the entire

communication phase. With the communication phase making up 10-90% of the epoch duration [39], delays in communication directly translate to increase in epoch duration and the overall training time, and may even lead to underutilization of expensive GPU resources [17, 18]. As a result, developing mechanisms to minimize (or evade) in-network congestion for AI training jobs, in order to speed up their communication phase, has emerged as an active area of research [5, 9, 10, 34, 47, 51, 52, 65].

However, most prior work in this area focus on dedicated clusters that are used and managed by a single entity, and where one can assume complete knowledge and control over all jobs running in the cluster, and also control over the underlying network infrastructure. An often overlooked setting is where distributed training jobs belonging to different users (or tenants) simultaneously run on a shared cluster (e.g. a large public cloud or a small-scale academic cluster). In such settings, network flows for a given training job can experience congestion from external sources (e.g. jobs belonging to other tenants, background data-transfers and storage tasks, etc) [27]. Schemes designed for dedicated clusters (e.g. evading congestion by coordinating the temporal schedules of different jobs [47] or the paths and priorities assigned to their flows [10]) are ill-suited in such shared settings: an individual tenant managing their own job has no control over other jobs belonging to a different tenant, or over the network infrastructure that is managed by the cloud operator. We also cannot rely on infrastructural mechanisms to dynamically route around congestion (e.g. adaptive routing in switches [12]), as they may or may not be supported by the shared cloud (the tenant has no visibility or control over it).

So what can an individual tenant, running their own distributed AI training job, do to evade congestion from external sources in a shared cloud? Motivated by this question, we build a system, *REACT* (short for Runtime Epoch-Adaptive Collective Tuning) that tunes the recurring *pattern* of communication among nodes in an AI job in order to alleviate congestion – changing the specific set of incident flows (source and destination nodes) while retaining the semantics of the required information exchange. REACT can be unilaterally deployed by an individual tenant to improve their own job’s performance – it works solely at the application (communication library) layer, and requires no explicit support from the network infrastructure and no control

over external workload. It also makes no assumptions about the underlying network protocols (e.g. RDMA, TCP, etc), and is complementary to any transport-level congestion control mechanisms and routing strategies (e.g. ECMP, adaptive routing, etc) deployed by the cloud operator.

Before explaining how *REACT* works, we provide some relevant context. Communication among the nodes in a distributed training job is specified through *collective* operations that capture the high-level semantics of information exchange between nodes. Examples of such a collective operation includes “AllGather” where a piece of information (e.g. model weight or gradients) at each node must be sent to all other nodes, and “AllReduce” where the piece of information at all nodes must be aggregated and the result must be disseminated to all nodes. AI training jobs commonly use a collective communication library or CCL (e.g. NCCL [11], Gloo [15], OpenMPI [1]) that executes the specified collective operation as a concrete sequence of data exchange steps among the participating nodes. We refer to this sequence of steps as the *collective pattern*. A given collective operation can be realized using different collective patterns. For example, an AllReduce between three nodes (N0, N1, and N2) can be realized using a tree pattern, where data from N0 and N1 is aggregated at N2 and the result is sent back to N0 and N1. The same operation can also be realized using a ring pattern where N0 sends its data to N1, which then aggregates it with its own data and sends the result to N2; N2 then aggregates the result with its data and sends the final result to N0 which then relays it to N1.

The CCL computes the collective patterns once at the start of the job (typically based on parameters such as number of nodes, message sizes, underlying transport protocols, etc) and uses the same pattern repeatedly in each epoch throughout the training run. Recent work proposes using additional information (e.g. profiled link characteristics, global knowledge of workload etc [5, 9, 52, 62]) to optimally compute the collective pattern at the start of the job. However, this initial collective pattern starts deviating from optimality as the network state changes over time (e.g. due to congestion caused by unforeseen arrival of external traffic).

Our system, *REACT*, tunes the collective pattern at runtime in response to congestion. As a simple example, for the tree-based AllReduce with three nodes, if the incoming link at N2 is congested, rather than aggregating all data at N2, we can aggregate at N0 or N1. In the ring-based AllReduce example, if the path from N0 to N1 faces congestion, we can swap the positions of N1 and N2 in the ring, to avoid that path. In contrast to adaptive routing mechanisms that tune flow paths to evade congestion, *REACT* changes the set of incident flows themselves (i.e. changes the source and destination nodes). It can therefore even help evade congestion on links that have no re-routing alternatives (e.g. the link from a ToR switch to a server [38]).

REACT runs a feedback loop at the CCL level, where it collects the completion times of all flows after a small set of epochs, analyzes them to detect presence of congestion, and then tunes the collective pattern in response to the detected congestion. Realizing such a feedback loop in practice requires tackling

multiple challenges:

(1) *How do we reliably detect congestion using only flow-level stats?* For this, *REACT* incorporates a mechanism for strategically comparing the completion time of various flows in a collective pattern with one another, and across epochs, to identify flows that potentially experience congestion, without assuming any knowledge about the underlying network topology or link capacities.

(2) Upon identifying the flows that experience congestion, *REACT* iteratively picks such flows that lie on the critical path of the collective pattern, and attempts to swap their source and destination nodes with other nodes in the collective pattern in order to alleviate congestion. *How do we determine which nodes can be swapped in a given collective pattern, such that we retain semantic correctness?* For this, we outline a mechanism for computing viable *transforms* for any given collective, that provides a set of viable nodes that any given node can be swapped with in the collective pattern without violating semantic correctness. We further prune this set to eliminate nodes that would not help alleviate congestion.

(3) *How do we ensure that REACT can update the collective pattern fast, within a few epochs?* *REACT* tests each transform to check whether it helps improve performance (and must be retained) or whether it should be reverted. Doing so one at a time by re-running the updated collective pattern in the next epoch after each transform will increase convergence time. To speed up convergence, *REACT* uses an analytical model to estimate the performance impact of a transform without actually running the updated collective pattern. This allows *REACT* to iterate through a batch of multiple transforms within the same epoch. We provide more details on each of these design aspects in §5.

Note that *REACT* can only respond to congestion that lasts for a few training epochs (i.e. for a duration of a few hundreds of milliseconds), e.g. from long-running data transfer tasks or other competing AI jobs. It is not meant to react to ephemeral congestion that dissipates within an epoch. Further note that our contributions are orthogonal and complementary to how the initial collective pattern is generated in the first place. Finally, it may not always be possible to eliminate *all* congestion by swapping nodes – *REACT* is a best-effort system that attempts to alleviate congestion to the best extent possible.

We prototype *REACT*, implementing it as a shim layer over PyTorch and NCCL. We evaluate it on a national academic shared GPU cluster, and show how enabling *REACT* improves the communication time by 13%-38% under different degrees of congestion. We further evaluate *REACT* by simulating a wide variety of congestion and network failure scenarios in ns3 [49] to show up to 75% improvement in algorithm bandwidth. Our evaluation spans a variety of popularly used collective patterns: NCCL’s dual binary tree based AllReduce [26, 50], Ring AllReduce [43, 58], and AllGather with recursive doubling [42, 58].

2 Background

Repeated rounds of information exchange. A typical AI training job runs over multiple rounds or epochs, where model

weights and parameters are repeatedly updated using iterative gradient descent. To scale training, the job is typically split between several nodes (or GPUs) by sharding the training data, model, and/or the tensors [32, 40, 48, 54]. Each epoch in such a distributed training job therefore comprises of a combination of local computation at each node, and communication phases where relevant data (gradients and updated weights) are exchanged among nodes. With each epoch repeating the same gradient descent logic, the same high-level information exchange between nodes is repeated in each epoch. These epochs therefore provide a useful time granularity for a reactive feedback loop, which is unique to AI/ML training, and is exploited by our system.

Information exchange specified via collective operations. The repeated information exchange between the participating nodes is specified at a high-level through collective operations (e.g. AllGather, AllReduce, etc, as mentioned in §1). Application-level libraries such as PyTorch [14] or Tensorflow [20] use the model specification and the set of participating nodes to generate a directed acyclic graph (DAG) of computation and communication tasks, schedule them across devices, and specify the communication requirements using collective operations. For the latter (i.e. for carrying out the communication tasks), these application libraries invoke a collective communication library (CCL), such as NCCL [11], Gloo [15] and OpenMPI [1], through standard collective APIs.

Collective operations realized via collective patterns. Given the collective operation, CCL computes the specific collective algorithm to realize the operation, i.e. how the data should be chunked up, and the concrete series of message exchanges among participating nodes. We refer to this series of message exchanges as the *collective pattern*. A given collective operation can be realized through several different patterns (all resulting in the same high-level information exchange needed at each training epoch). For example, AllReduce can be accomplished using a tree pattern or a ring pattern (as exemplified in §1), as well as other patterns. AllGather can be accomplished through broadcast by each node, or through a more structured recursive doubling pattern [58]. We provide more details about these popularly used collective patterns in Appendix A.1, and briefly discuss how CCLs select collective patterns in the next section.

The CCL executes the collective operation by invoking the underlying transport (PCIe or NVLink for intra-host communication, RDMA or TCP/IP for inter-host communication, etc) and setting up the message exchanges between the corresponding source-destination pairs, as determined by the collective pattern. The collective patterns typically have a hierarchical structure, where the intra-host data exchange is handled first, followed by inter-host data exchange. In this work, we focus on the inter-host component of collective communication (discussing the intra-host aspects in §8).

3 Related Work

3.1 Computing Collective Patterns

Heuristic-based Collective Patterns. Using communication collectives to exchange data is an extensively studied field in HPC and ML communication, and different heuristics have been developed to compute collective patterns for specific network topologies and stacks [26, 42, 43, 50, 51, 58]. For instance, NCCL, the most widely used CCL for distributed training, maintains a fixed set of patterns for each collective operation (e.g. a dual binary tree and a ring for AllReduce), and selects the lowest cost pattern for the given setting, where cost is computed based on the message size, number of nodes/devices (GPUs, NICs) and pairwise link bandwidth and latency (that are hardcoded for each underlying protocol – RDMA, TCP, GPUDirect, SHARP, NVLink, PCIe, etc).

Solver-generated Collective Patterns. Several recent works (e.g. [5, 9, 52, 62]) propose finding the optimal collective pattern by modeling the underlying network and workload characteristics in solvers like Z3 and Gurobi, with the objective of reducing the overall communication time. While such approaches promise more optimal outcomes than heuristics, they are computationally expensive to run repeatedly. Additionally, they require complete knowledge of link characteristics and all workloads in the cluster to compute the optimal collective pattern, which is not always possible.

Both of the above approaches generate a collective pattern once at the start of the job, and use it repeatedly over the subsequent epochs. Our work takes such a pre-computed collective pattern as input, and minimally tweaks it at runtime in order to react to congestion.

Adapting Collective Patterns. There have been a few proposals to adapt communication collectives at runtime. Plink [34] periodically probes all pairwise network paths, and uses the resulting bandwidth and delay information to choose the root and leaf nodes for AllReduce operations realized via two-level hierarchical trees. However, such constant probing is costly and cannot be done at small enough timescales (especially when frequent re-profiling is required to capture the variance that may arise from probabilistic ECMP hash collisions). Moreover, Plink adaptation is restricted to how the root and leaf nodes are chosen in the hierarchical tree. In contrast, our goal is to provide a more general reaction to congestion for any given collective pattern.

Another work, AutoCCL [63] also tunes collective communication at runtime, but focuses on only tuning pre-defined NCCL parameters (e.g. the chunk size, number of channels, threads, etc.) without changing the intrinsic communication pattern itself.

Another closely related work is AdapCC [65], that also periodically probes all pairwise network paths at coarse timescales of every five hundred epochs, and uses a solver to recompute the collective pattern based on the updated information about network delays (running such a solver at smaller timescales is prohibitively expensive). In contrast, our system, *REACT*, works at a much smaller timescale of a few epochs that is complementary to

AdapCC – *REACT* can use the infrequently generated collective patterns by AdapCC as inputs, and minimally tweak them to provide an order of magnitude faster reaction to congestion.

3.2 Minimizing Network Delays in Distributed Training

The standard way to react to network congestion is for the congestion control algorithm used by the underlying transport (e.g. TCP [25], RDMA [66], or customized transport [17, 24, 31, 56]) to kick in and reduce the sending rate of the flow. However, this still increases the flow completion time, and thereby the training time. Therefore, several solutions have been proposed to evade congestion during AI training.

Routing-based Solutions. One of the proposed solutions is to avoid congested paths by rerouting flows using infrastructural support, e.g. adaptive routing in switches [12], or source routing via port changes in the hardware transport or the NIC (that determine ECMP hashes) [8, 10, 16, 28, 37]. Another alternative is to avoid creating hotspots in network links by leveraging switch support to uniformly spray packets [4, 7, 29, 33]. Such infrastructure-level solutions are beyond the control of individual tenants using a public cloud cluster (they may or may not be supported or enabled). Moreover, re-routing cannot help when congestion happens on an unavoidable link with no alternatives (e.g. the link from a ToR switch to a server [38]).

Coordinated Scheduling. Another set of solutions minimize the effects of congestion among competing AI jobs by coordinating their spatial or temporal schedules. For instance, Cassini [47] develops a global scheduler that computes the temporal schedule of all jobs in the cluster to minimize overlap in their communication phases. Crux [10] coordinates the path selection and priorities across all jobs to reduce the communication time that would result in idle GPU. Both of these approaches assume control over the entire workload (which is beyond the control of an individual tenant in a shared cloud), with Crux additionally assuming infrastructure support for in-network telemetry, source routing, and prioritization.

Another body of work attempts to reduce communication delays using a best-effort co-location strategy to place a given job on closeby machines [35, 46] – these work are orthogonal and complementary to *REACT*, which attempts to alleviate inter-host congestion for a given workload placement.

Application-Integrated Approaches. Another orthogonal and complementary body of work attempts to minimize the effects of communication delays by increasing the overlap in computation and communication phases [36, 44, 64], or reducing the data to be communicated via compression or quantization [36, 44, 64], or dropping low-information packets during communication to reduce tail latencies [59, 61]. These work require tighter integration with the AI application and are not semantically transparent. *REACT*, in contrast, works at the communication collective library layer, without requiring any modifications in the overlying application (or the underlying transport and network infrastructure).

4 Overview

4.1 Key Design Considerations

Target Scenarios. We design *REACT* for shared GPU clusters, where distributed training jobs from one user (or tenant) can face external congestion from competing distributed AI workloads from other tenants, large file transfers, or even the other communication tasks within the same training job like check-pointing or copying training data from storage servers to GPUs. These congestion scenarios can last for several hundreds of milliseconds or even longer [19], providing time to react over timescales of a few training epochs.

Our targeted settings may range from large high-end clusters in the cloud to smaller-scale shared academic clusters. These clusters may vary in network topology (large-scale Clos networks connecting thousands of servers vs small-scale star topology on a single rack), the underlying network protocols (different variants of RDMA, TCP/IP, or proprietary solutions [13, 24, 31, 56], ECMP vs adaptive routing, etc), server configurations, and so on. These infrastructural aspects are determined by the cluster operators, with individual tenants having limited or no control over them. We design *REACT* to work across all of these various infrastructural settings and underlying network protocols. *REACT* can be unilaterally deployed by individual tenants or users to alleviate congestion for their own AI training job within a few training epochs, irrespective of how the underlying network is managed by the cluster operators. This requires us to tackle the following design challenges:

Challenge 1: Only application-layer changes. We cannot assume any network support, or make low-level infrastructural changes (such as tuning the network paths traversed by flows [8, 10, 12]), as these are beyond the control of individual tenants. To enable individual tenants to unilaterally deploy *REACT*, it works solely at communication collective library (CCL) level. Specifically, *REACT* uses flow-level stats readily available at the CCL to detect congestion. It then reacts to congestion by changing the collective pattern that is executed by the CCL. Deployment options for *REACT* can range from incorporating *REACT* logic into existing CCLs (NCCL, Gloo, OpenMPI, etc) or deploying *REACT* as a separate higher-level library over an existing CCL (as done in our prototype implementation in §6).

Challenge 2: No global cluster information. We assume that each training job runs independently and has no visibility into other concurrently running jobs (potentially from different tenants) or the global network state. This restricts *REACT* to only using information that is locally available at the CCL for a given job. *REACT* runs a reactive feedback loop system based on this local information. This is in contrast to solutions such as Cassini [47] that assumes knowledge about all jobs running in the cluster to co-optimize their schedule.

Challenge 3: Low overhead. We design *REACT* to be a low cost solution that users can easily deploy, and that can react within timescales of a few epochs. To that end, we eschew the use of explicit link profiling for determining network conditions (as

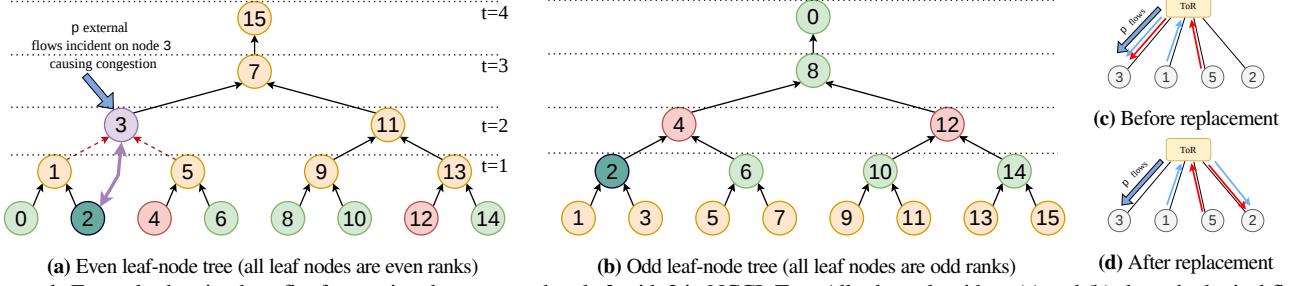


Figure 1: Example showing benefit of swapping the congested node 3 with 2 in NCCL Tree-Allreduce algorithm. (a) and (b) show the logical flows between nodes in Tree-Allreduce. (c) and (d) show how the congested physical link is avoided by interchanging positions in the logical tree algorithm (the red and blue arrows are the chunks sent from 1 and 5 to 3 before and to 2 after replacement).

in [34, 65]), instead relying on flow-level stats readily available at the CCL. Moreover, we choose to use simple heuristics to minimally tweak the given collective pattern in response to congestion rather than using expensive techniques to recompute the entire pattern (as in AdapCC [65]). Tweaking the collective pattern minimally also helps minimize the overheads of updating the pattern (which can require setting up new connections) and further try to maintain their desirable properties (like latency or bandwidth optimality).

4.2 REACT in a nutshell

REACT uses a feedback loop to explore and tune the given collective pattern in response to congestion, based on the current network state. At the end of each training epoch, *REACT* collects the flow completion times (FCTs) of individual flows from each node that participates in the collective pattern, and sends them to the leader (a randomly selected node among all participating nodes). After a small batch of epochs (set to 6 in our implementation), the *REACT* logic running at the leader analyzes the FCTs to detect presence of congestion and to identify potential sources of congestion. It then modifies the collective pattern by applying a series of *transforms* in order to circumvent congestion and improve performance. The modified collective pattern is then disseminated to all participating nodes and applied for the next batch of epochs. This process repeats until no further congestion is detected, or no further changes help improve performance. Our transform logic is designed to ensure that all modifications made to the collective pattern are *safe* i.e. any modification preserves the communication semantics expected by the application, resulting in the same logical data exchange as required by the collective. §5 provides details of how the feedback loop works, how *REACT* detects congestion, and how it transforms the collective pattern.

4.3 Illustrative Example

We highlight the potential of our approach through an illustrative example. Our example considers the Tree-based AllReduce collective operation for a 16 node distributed training job. We consider the collective pattern generated by NCCL as the starting point, that *REACT* adapts in response to congestion. NCCL constructs two binary trees over the 16 nodes (referred as ranks 0-15), and divides each message into two chunks to be separately reduced using the two trees. Fig 1a and 1b show the two binary

AllReduce trees constructed using NCCL’s heuristics. The trees are constructed such that the leaf nodes in one tree all have even ranks (Fig 1a), while those in the other tree all have odd ranks (Fig 1b). The directed edges in the tree show the flows to be completed for the all-reduce: once each node receives all chunks from lower layers, it aggregates them before forwarding the result to the higher layer node. The reduce part of the collective is completed when each of the chunks has been aggregated at their respective root nodes (rank 15 and 0). In this example, it takes four time steps to complete it (which correspond to the four layers in the tree). The broadcast phase is started after this by sending the information back along the reverse edges in the two trees.

For simplicity of exposition, our example assumes all 16 nodes are connected to the same switch (or ToR). Fig 1c shows a subset of these nodes (other nodes not shown for brevity). Suppose the link propagation delay is α and the link capacity is $\frac{1}{\beta}$.

Performance without congestion. We first consider the scenario where there is no external congestion, and compute the time taken for the reduce operation in terms of α and β [23]. Assuming the capacity at each link is fairly divided among the flows incident at the link, the total cost to reduce a message of size s will be: $4\alpha + \frac{7}{2}\beta \cdot s$. (Explanation provided in the footnote.)¹

Congestion degrades performance. We next consider the case where there are p external flows incident on node 3, reducing the fair share of flows incoming at node 3. This delays the data aggregation step at node 3, and the delay cascades over to higher layers (as node 7 must wait for input from node 3 before doing its aggregation, and so on). The cost for the reduce phase now will be $4\alpha + (3 + \frac{p}{2})\beta \cdot s$. (Explanation provided in the footnote.)²

REACT adapts the collective pattern to avoid congestion. Once *REACT* detects incoming congestion at node 3 (as detailed in §5.2), it attempts to swap nodes such that congestion is alleviated. Specifically, node 3 has two incoming flows in the current collective pattern— *REACT* would find a swap that would reduce the number of incoming flows at the congested node 3, while still retaining the semantic correctness of the AllReduce operation. In Fig 1a, the only “viable” replacements for node 3 are the green nodes. Interchanging with the yellow nodes will not

¹The latency term comes from the 4 steps and bandwidth term for each of the steps are summed together: $(2+2+2+1)\beta \cdot \frac{s}{2}$ as bandwidth is shared on first three steps and message size is divided by two for each chunk.

²Similar to previous calculation: $4\alpha + (2 + (p+2) + 1 + 1)\beta \cdot \frac{s}{2}$.

help alleviate congestion as even after replacement, node 3 will still receive two incoming flows. The red nodes 4 and 12 are not viable because e.g., if 3 and 4 are interchanged in the even leaf-node tree (Fig 1a), then it is highly likely that the incoming flows to node 4 from nodes 1 and 5 in even leaf-node tree (Fig 1a), and 2 and 6 from the odd leaf-node tree (Fig 1b) will compete for link capacity. We discuss in §5 how the set of viable nodes can be identified more generally for a given collective pattern.

In this case, we select node 2 randomly from the viable set of green nodes to be swapped with node 3. This allows us to reach a configuration as displayed in Fig 1d where the extra cost due to congested flows can be avoided. Note that we do not modify the odd leaf-node tree in this case as the congested node 3 is already a leaf node there. This approach can be extended to handle more congested nodes by repositioning them on their corresponding trees. We can make similar optimizations on the reverse broadcast path for outgoing congested flows.

For a typical $\alpha = 4\mu s$, $\beta = 1/(100\text{Gbps})$, $s = 2\text{MB}$, and $p = 2$, the time to complete the reduce phase increases from $576\mu s$ under no congestion to $656\mu s$ due to congested flows. The swapping of nodes helps in reverting the reduce phase time back to $576\mu s$ despite congestion— a 14% improvement (stronger performance wins can be seen in §7 as we vary the settings, e.g. the number of participating nodes, the degree of congestion, the position of congested node in the pattern, etc).

5 Design

We describe how *REACT* detects congestion at runtime in §5.2. In §5.3, we discuss how *REACT* pre-computes the set of safe transforms (viable swaps). Finally, in §5.4, we describe *REACT*’s runtime feedback loop.

5.1 Preliminaries and Notations

We represent a collective pattern (CP) as a time-expanded network (TEN) [6, 30, 57], where each vertex (t, r) corresponds to a rank (the logical id assigned to a GPU/compute node), r , at a logical time step, t , and each directed edge $((t, r) \rightarrow (t+1, r'))$ corresponds to a flow executed between two ranks in consecutive steps.

Fig 2 shows the complete unrolled TEN graph of NCCL’s Dual Binary-Tree AllReduce discussed in the example in §4 (this differs from Fig 1 in that it also includes the broadcast phase of AllReduce). We additionally discuss Ring AllReduce and Recursive Doubling AllGather and their respective TEN graphs in Appendix A.1 (Fig 9a and Fig 9b). We focus on these three collective patterns as our case-studies. Nonetheless, *REACT* is a general system that can be used to tune other collective patterns.

5.2 Detecting congestion

After running a batch of B epochs, *REACT* collects the flow completion times (FCTs) of all flows F in the CP in each epoch. Let $\mathbf{b}_f$ be the vector of the observed throughput (calculated using FCTs) in the most recent batch of B epochs for a particular flow f . We now discuss how *REACT* uses the TEN digraph G and throughput vector of all flows $[\mathbf{b}_f]$ to detect which

flows experience congestion, and the type of congestion. We differentiate between two types of congestion:

(1) Steady congestion. This type of congestion is persistently experienced by a flow in each epoch. It can arise when an unavoidable link (with no other multipath alternative) experiences congestion. For example, the link between a server and the next hop switch (typically, the top-of-the-rack or ToR switch) can face congestion due to multiple other flows running on the same server in parallel with the training job. These could be the storage traffic for checkpointing, data copy, and other such CPU-centric workloads [18], or other AI jobs scheduled on the same server (e.g. due to fragmentation of resources [10, 35], GPU sharing [60], etc).³ Steady congestion can also arise if the spine layer of the underlying network topology is so heavily congested, that all paths through the spine layer experience performance degradation [19].

Steady congestion is characterized by high FCTs for a flow in all epochs. We therefore identify whether a flow f is *steady* congested by checking

$$p90(\mathbf{b}_f) < \text{mean}(\mathbf{b}_{F(k_f)})(1 - \delta_1) \quad (1)$$

where $\text{mean}(\mathbf{b}_{F(k_f)})$ is the mean throughput of all observed flows that have the same degree k_f as of flow f in the TEN graph (specifically, we consider the maximum of in-degree and out-degree). The $p90(\mathbf{b}_f)$ is the 90th percentile value of the observed throughput for flow f . Eq. 1 checks whether flow f performs worse than other similar flows in the CP for majority of the epochs. For our experiments, we set δ_1 to 0.2 as we expect a performance drop of at least 20% under congestion for it to warrant attention. This threshold also helps avoid noise due to changes in the underlying stack and can be finetuned based on deployment.

Erratic congestion. Congestion also commonly arises in other links within the network that may not always lie on the flow’s path in each epoch due to other multipath alternatives. For example, congestion arises in the spine-to-ToR links in a typical fat-tree topology in large-scale training jobs spanning multiple server racks [17, 45, 55]. Due to non-deterministic behavior of the underlying multi-path routing strategies (e.g. ECMP or adaptive routing), a flow may encounter a congested link on its path in one epoch, but not in another. We characterize the resulting congestion, observed only in a subset of epochs in the batch, as erratic congestion. We identify whether a flow is erratic congested by checking

$$\text{normalizedSTD}(\mathbf{b}_f) > \delta_2 \quad (2)$$

where $\text{normalizedSTD}(\mathbf{b}_f)$ denotes the normalized standard deviation of observed throughput. We set $\delta_2 = 0.15$ in our experiments.

Outside of congestion, another form of network performance degradation can arise due to partial link failures, e.g. where performance of a link drops from say 100Gbps to 30Gbps due to a switch dropping packets at a particular rate [7, 22]. *REACT*

³Chances of congestion on server-to-ToR links are lower in high-end deployments with over-provisioned servers that have dedicated NICs per GPU. Our work generally targets a wide-range of settings (including academic clusters and other low cost deployments) where the same NIC could be shared across GPUs and by storage traffic, leading to potential congestion in server-to-ToR links.

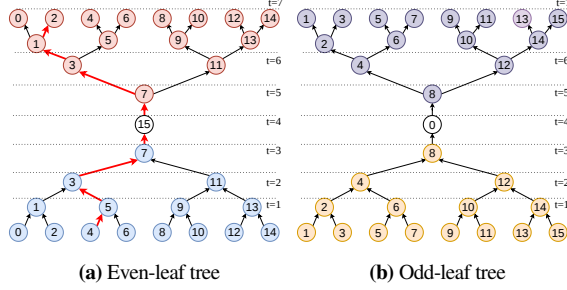


Figure 2: TEN of NCCL dual-tree AllReduce

naturally extends to such failures too, treating them similar to a congested link. Such network failures are characterized as steady congestion when they arise in unavoidable server-ToR links, and as erratic congestion otherwise.

5.3 Finding Safe Transformations

Next, we describe the transforms— the replacement strategies— used by *REACT* to avoid congested flows. We start with the TEN digraph for the given CP and detect congested flows. Once we detect a congested flow, we mark its source and destination as problem nodes. We want to avoid traversing this flow in the collective graph in subsequent epochs. To resolve congestion, we select one of the problem nodes and swap it with other nodes in the TEN graph. We now discuss how these swappable nodes are selected.

5.3.1 Finding semantically viable swaps

We want to swap nodes while ensuring that the new pattern (represented by its TEN graph) retains the semantics of information exchange for the collective operation. We discuss three swapping transforms that ensure this.

① **Global Permutation swap:** The main idea is derived from the observation that most popular “All” collective operations (AllReduce, AllGather etc) are equivalent under permutation of ranks, in that we can swap all instances of a device x with all instances of device y in the TEN graph, and the pattern still remain semantically equivalent. Note that this transformation is similar to finding an appropriate permutation of rank to device mapping.

We can apply this swap in a dual tree transform (Fig 1) by replacing all instances of congested node 3 with all instances of node 2 in both trees (Fig 1a and 1b). If we face erratic congestion (spine-TOR links) in a flow of the dual-tree collective, this transform helps avoid that particular spine link by changing source/destination. However, if the congestion is in the server-TOR links, the performance deterioration persists as the overall degree of congested node 3 remains same before and after the swap, across the collective. Note that the transforms have different effects depending on the initial CP— we discuss ① in the context of other collectives in §5.3.3.

② **Chunk-local Permutation Swap:** Typical CPs divide the message to be transmitted into chunks. Each chunk is transmitted independent of the other chunks, e.g. the even-leaf and odd-leaf trees in Fig 1 transmit two different chunks separately. We identify independent chunk graphs in a CP and apply the *Permutation Swap* transform selectively on only the affected

chunk graphs. Fig 2 shows the unrolled TEN digraph for the dual-tree AllReduce algorithm, where we can swap nodes in both trees independently. E.g., if node 15 in Fig 2a faces congestion, its effects might be alleviated if all of its instances in Fig 2a are swapped all instances of an uncongested node 10, without modifying any nodes in Fig 2b digraph.⁴

It is more desirable to use such a finer grained swap (as compared to global permutation) when possible, as it allows making selective changes, local to wherever performance deterioration is detected. Making more changes than necessary can add to the overhead in terms of exploring more unused network paths.

③ **Position-equivalent Swaps:** The above two transforms are effectively permutation of rank to device mapping applied on the collective pattern and chunk-graph scale respectively. Next, we apply a brute force algorithm (formally defined in Appendix A.3) on each chunk graph to identify the equivalent positions that can be swapped while maintaining semantic equivalence of the CP. The algorithm considers all pairwise positions and checks whether swapping them violates the original collective operation. We check this by assigning all nodes a bitvector with a single bit set to 1 based on rank. We simulate an execution of the collective operation on this vector by running a breadth-first traversal of the collective graph and checking whether the final bitvector on all nodes aligns with the conditions of the collective operation. For the popular collective patterns, the brute force algorithm has an $O((N \log N)^3)$ complexity to find the chunk graphs and equivalent sets of swappable nodes. We only generate this once at the start of the job and can refer to it at runtime.

Fig 2 shows the equivalent positions in dual-tree AllReduce algorithm where the nodes with same color form an equivalent set and can be freely swapped.

Note that the search space of transform ③ is different than the previous two transforms. This transform allows even finer grained changes than ① and ② that help make precise decisions in resolving congestion. Eg, if some node is steady congested, we only want to swap its placement wherever it lies on the critical path. Changing other positions may not help improve the collective completion time but can add additional uncertainty. Based on this granularity of changes, for a given congested node, we prioritize available ③ transforms over ② and ①.

5.3.2 Pruning the Swap Space

Given a congested node, we get a set of transforms from above that can be applied to possibly alleviate congestion. We further prune this set by checking that after applying the transform, for the complete CP, the degree of each rank at various timesteps does not exceed the degree of the original rank (before the swap) in this position. In the example in Fig 1, this check removes all the red nodes from the set of viable nodes. For the given congested node, this pruned set is the set of viable transforms we have available (yellow and green nodes in Fig 1a). Among the

⁴The swap can help alleviate erratic congestion between node 7 and 15. In the case of steady congestion, it removes 15 from being the bottleneck of the pattern and shifts it to a location where it is less likely to affect.

Transforms	Ring	Recursive D.	Tree
①	Spine, Failure	All	Spine, Failure
②	✗	✗	All
③	✗	Spine, Failure	All

Table 1: Comparing the types of congestion (§5.2) resolved by the transforms for popular CAs (Ring-based algorithms, Recursive Doubling AllGather, Tree based algorithms). *Spine* is Spine-ToR congestion, *Failure* is network failures, *All* is all congestion types in §5.2. (✗) marks that the transform is not able to help with the CP, topology and congestion scenarios we consider.

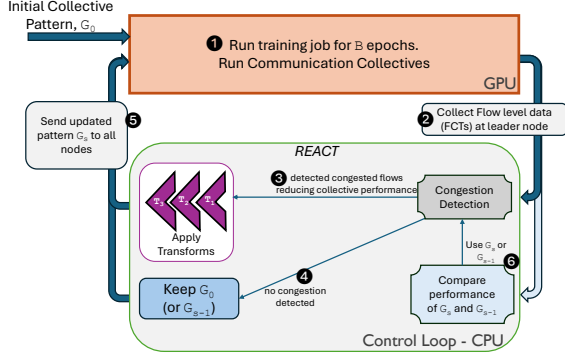


Figure 3: State diagram of *REACT*. The orange box marks the distributed training workload running on the GPUs that uses the input collective pattern for communication. The gray boxes run on a separate thread on the CPU—collecting flow level data from all nodes, running the control loop, finding a new collective pattern, and sharing it with the training job processes on all nodes.

set of viable transforms, we further prioritize swaps that reduce the degree of congested nodes (the green colored nodes in Fig 1). We randomly choose a node to swap from this prioritized set (e.g. node 2 in Fig 1). If none of the prioritized nodes help improve performance (as discussed in §5.4), we consider transforms with other viable nodes (i.e. the yellow nodes in Fig 1).

5.3.3 Applying transforms on collectives

While we exemplify the above transforms in the context of AllReduce trees, they are broadly applicable. We also apply and evaluate these transforms for Ring AllReduce and Recursive Doubling AllGather (these collective patterns have been explained in Appendix A.1). The applicability of *REACT*’s transforms depends on the structure of the underlying collective and the type of congestion. Table 1 summarizes the applicability of the transforms to the three transforms we study and the different congestion scenarios. Tree AllReduce supports all three transforms; Ring AllReduce primarily benefits from global permutation, since changing node positions within individual chunks does not reduce node degree; and Recursive Doubling supports global permutation and position-equivalent swaps. We provide detailed examples and analysis for each collective in Appendix A.2. Other popular collectives like ReduceScatter, Broadcast, Reduce, Scatter etc. have similar collective patterns and our work can be applied for them as well.

5.4 Runtime Feedback Loop

Fig 3 shows the state diagram of *REACT*’s runtime feedback loop. At the start of the job, we take the initial collective graph G_0 . We run the training job with G_0 and once a batch of B epochs is completed (①), all flow completion times (FCTs) in the CP are collected at the leader node (②). We discuss more on how the collection and sharing of FCTs is implemented in §6. Next, we identify the congested flows using methods from §5.2. If no congestion is detected (③), the control loop decides to keep the current CP graph. If congestion is detected (④), we proceed to applying viable transforms on G_0 . The updated graph, G_1 , is shared with all nodes by the leader node (⑤). We explain these steps in §5.4.1. *REACT*’s feedback loop then repeats these steps, taking the updated graph G_1 as input to produce the next graph G_2 , and so on (as detailed in §5.4.2).

5.4.1 After a single batch of epochs

We now discuss how, after running a batch of B epochs with an input pattern G_0 , we apply transforms to G_0 and swap out congested nodes to get an updated pattern G_1 . The algorithm pseudo code can be found in Appendix A.5.

We begin with allocating weights to each edge in the input TEN graph, G_0 , based on completion time of each flow in each epoch in B . We then find the critical paths in each of these weighted graphs, where the critical path is the one with the highest edge weights (total completion time). We also construct another graph where the edge weights are derived from the average FCTs across all epochs in batch B , and compute its critical path. Reducing the time taken along the critical path will help reduce the overall collective communication time. Therefore, we only consider swapping out congested nodes that lie on at least one of these critical paths. Note that we construct critical paths for each epoch individually (along with looking at the average FCTs across epochs) because erratic congestion would impact only a subset of epochs.

We want to prioritize resolving *steady* congestion as it is more severe than *erratic* congestion. We can further identify whether a node is steady congested by checking if all incoming/outgoing flows to a node in the graph are *steady* congested. Thus, we sort all vertices $x = (t, r)$ in G_0 based on the number of congested flows that have the vertex as a source or destination. We iterate over the list and for each congested node, find a viable list of transforms (as detailed in §5.3).

If multiple transforms with same priority are available, we randomly select a replacement for the congested node among the viable nodes (§5.3.2), to get a new collective pattern, G'_0 . We also update the initial $(B+1)$ critical paths (average and per-epoch in B) by applying the transform on each of them as well. We then estimate the performance improvement offered by this transform using a simple analytical model. We estimate the FCTs (i.e. the weight of each edge) in G'_0 based on mean observed FCTs for the corresponding source-destination pairs over the previous batches of epochs, scaled as per the new node degrees and message sizes.

In case a transform introduces flows for which we do not have past data, we estimate its performance as the median completion time across all flows of same degree. We only accept the updated graph for the next iteration if the mean of path times of updated critical paths is at least $(1 + \varepsilon)$ times lower than the original.⁵ In our experiments, we set ε to 5%.

Note that once we have resolved the *steady* congestion with a transform above, we remove these steady congested nodes from the list of viable nodes for other congested nodes, so that they may not reintroduce congestion in future transforms. Erratic congested nodes can still be reused, provided they pass the ε check above.

After each batch of epochs, we try to apply a maximum of K_t transforms to ensure we explore for a bounded amount of time. We set K_t to 15 in our experiments but increase it every batch of epochs so that the search space expands in later epochs. We obtain the updated graph G_1 , after applying this set of transforms.

5.4.2 Over multiple batches of epochs

Once a new graph has been generated after the above process, we run the next batch of epochs with the modified pattern G_1 (❶). After the completion of the new batch of epochs, *REACT* compares the performance of the current modified CP, G_1 , with the previous one, G_0 (❷). If the observed epoch communication time with G_1 is not less than $(1 + \tau_r)$ of the previous batch of epochs with G_0 , then *REACT* reverts to the older CP G_0 and tries new transforms on it, following the algorithm in §5.4.1. Else, we try to resolve congestion in G_1 , if any, following the steps in §5.4.1. We set τ_r as 0.05 for our experiments as we expect each modification to provide at least 5% improvement. Note that this check is based on the observed empirical performance, and differs from the check made for each transform in §5.4.1 that was based on estimated (analytical) performance.

We use this process to continuously explore for newer collective patterns and reach a more desirable solution (alleviating as much congestion as possible). It is possible that we may have reached a good enough solution and not want to explore any further. Hence, we set E_{max} , to 5, as the number of exploratory rounds (where each round has B epochs), after which we stick to the current CP if no better pattern was found. We set the batch of epochs size, B to be 6 in our experiments. Moreover, if no exploration was done over E_{max} rounds (i.e. $E_{max}B = 30$ epochs), we trigger exploration in the next round, to ensure we are not stuck in a local optima.

Since a cluster can have varying jobs and network performance, we delete all old FCT-data from beyond the prior H epochs. This helps account for any job churn in the cluster and we set it to 60 epochs for our experiments.

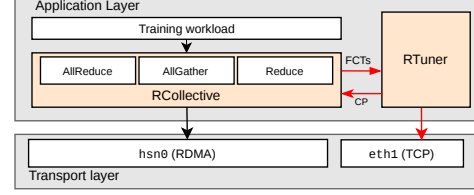


Figure 4: *REACT* architecture.

6 Implementation

We implement *REACT* at the application layer with two components: a collective execution library (RCollective) and a runtime tuner (RTuner). Figure 4 shows the architecture of *REACT*. We implement the collectives such as AllReduce, AllGather etc in the collective library. We implement the tuner to collect flow-completion statistics across nodes, compute an updated collective pattern, and disseminate it to all participating nodes.

6.1 RCollective library

We implement the RCollective library as a wrapper using the peer-to-peer (p2p) communication APIs— `isend` and `irecv` in PyTorch. Given a TEN graph of a communication collective, chunk sizes and specific intermediate aggregation steps, we implement RCollective to post corresponding communication kernels to the GPU asynchronously. We use separate sets of streams on the GPU for each chunk sub-graph of the collective algorithm so that maximum operations can run in parallel. We also ensure the data dependency between communication steps by adding blocking commands between the streams within a chunk sub-graph. We post all the kernels at the start of the collective and rely on the GPU kernel scheduler for efficiency. See Appendix B.1 (Algorithm 2) for the detailed pseudo-code for implementing a custom collective pattern using p2p APIs.

We also measure flow completion times by inserting timing events before and after each peer-to-peer operation. These events capture the elapsed time on the GPU stream until the transmitted data becomes available to the next dependent kernel.

Overheads. The RCollective library lies on the main performance path of the training workloads. The overheads due to multiple streams and flow timing events are minimal as these objects are created once at the start of the job and reused throughout the job lifetime. Other overheads occur due to setting up of new NCCL “communicator objects” when a new graph algorithm is sent by RTuner and missing out on the buffer management optimizations of NCCL. These overheads are discussed in more detail in §7.

Comparison with NCCL. A tighter integration with NCCL would likely reduce prototype overheads and provide a cleaner implementation path. In principle, NCCL’s ext-net profiler interface for custom network support could be repurposed as a

⁵Note that we compare the performance using old critical paths and choose not to recompute critical paths of G'_0 (for ε -check) as it is possible for a TEN graph G_0 to have multiple critical paths of similar completion times – we fix them iteratively.

lightweight wrapper around the underlying transport, with hooks to measure flow start and completion times. In practice, this was difficult in our prototype deployment because the shared academic GPU cluster we use relies on the Cray Slingshot network [24], whose transport stack is exposed through cluster-specific libraries (e.g., libfabric.so) and a NCCL ext-net plugin. This makes it difficult to insert our custom measurement hooks without modifying the underlying system software.

6.2 REACT Tuner

We configure the REACT Tuner (RTuner) component with one leader node and rest as workers. The leader is responsible for collecting all of the data from various devices, generating a new collective pattern based on the observed state, and sending the new graph to all other nodes. The worker processes send the measured flow completion times after every batch of B epochs and wait to receive the new graph from the leader.

We run RTuner on a separate CPU thread so as to minimize any interruptions to the main training process and it is initiated at the start of the job. The flow times and graphs are exchanged with RCollective library via shared memory. We run the leader RTuner process on rank 0 by default. We use the `gather` and `broadcast` communication collective operation in the Gloo [15] library to collect flow times from all nodes and send the new graph.

Overheads. RTuner primarily consumes additional CPU resources. In our setting, this overhead is small relative to the overall cost of distributed GPU training, since the dominant runtime cost remains GPU computation and communication. RTuner also uses a different network interface from the data path as discussed in §6.3.

6.3 Prototype on a shared cloud

We run our system on a national academic GPU cluster. Each server in the cluster has 4 A100 GPUs connected via NVLink. The servers are all connected by a 200Gbps Cray Slingshot [24] network `hsn` interface which is used by the RCollective to run communication. All nodes also support a Intel Corporation I350 Nic `eth1` interface which is used for cluster management and sending control messages.⁶ RTuner uses `eth1` as the interface to exchange control loop information via Gloo to avoid disturbing the collective performance. Further, to avoid making the main job wait for updates from RTuner, we let the main job continue for an additional 5 epochs with the older collective pattern. This allows RTuner to complete the consensus while the training job keeps running without any additional wait time. See Appendix B (Fig 10) for the intra-node and the Slingshot network topology.

7 Evaluation

We discuss our evaluation methodology in §7.1. Then, in §7.2, we present our evaluation on a small-scale testbed implemented on a national academic shared cluster (as described in §6). Finally, in §7.3, we present our larger-scale ns-3 simulation results under different scenarios.

⁶Many GPU clusters support such a configuration with 2 NICs- one for fast data path and second for slower control path.

#Nodes	p	Speedup $\times$		$\overline{\#changes}$
		Mean	p95	
4	0	0.964	0.999	34
	1	1.138	1.101	35.6
	2	1.348	1.145	33.2
8	0	0.963	0.980	41.25
	1	1.180	1.165	40
	2	1.383	1.222	36.25

Table 2: Ratio of mean (or p95) collective completion time over epochs for baseline to *REACT*, across node counts and congestion degree, p , for Tree AllReduce on GPU cluster deployment.

7.1 Evaluation Methodology

Collective Benchmark. We run collective operations in a tight loop over multiple epochs on a pre-defined set of nodes. We experiment with different collective operations – NCCL’s tree AllReduce, Ring, Recursive Doubling. We evaluate how long the collective communication takes over epochs with and without *REACT* under different congestion scenarios.

Modeling Congestion. We model congestion in three ways. First, we inject external background flows from nodes that are not participating in the collective into one or more nodes that participate in the collective. We define the congestion degree p as the number of external flows injected per affected source-destination pair. Second, we run two jobs simultaneously with a configurable number of overlapping nodes to model resource fragmentation and server-level contention. Third, we run two simultaneous jobs with no shared nodes but overlapping network paths to isolate network-only contention. The latter two are only evaluated in simulation, while the first congestion scenario is used both in our testbed and simulations.

Network Topology. For the testbed evaluation, we use the provided Slingshot topology (See Appendix B, Fig 10b). We further evaluate *REACT* in simulation on three network topologies: a star topology, a 128-server Clos topology [2], and a topology derived from a publicly available Alibaba trace [10, 21]. The Alibaba topology has 48 TORs, each connected to the 3 aggregate switches. Each server is connected to exactly one TOR. We simulate a 720-server Alibaba topology.

Metrics. We measure the total collective completion time per epoch and also calculate the algorithm bandwidth by dividing the message size by the collective time, as done in prior works [5, 52]. We compare the improvement in algorithm bandwidth (or reduction of collective completion time) of *REACT* against the baseline under the various scenarios.

7.2 Testbed Evaluation

We first show the feasibility of our approach and its benefits by implementing *REACT* on a shared cluster, as discussed in §6. We run two benchmark jobs with 4 and 8 A100 GPUs respectively (Table 2). We enable adaptive routing and GPU Direct RDMA in the underlying Cray Slingshot 200Gbps network [24] for our experiments. We run the dual binary Tree AllReduce for 5k epochs with a 100MB allreduce message size.

We request one GPU each on different nodes from the

cluster scheduler to evaluate *REACT* under inter-server network congestion. Since we cannot control the node allocation from the cluster scheduler in terms of node placement in the topology, we run our benchmarks multiple times to try different resource allocations. For fairness, we only make comparisons by running both the baseline and *REACT* on the same allocation one after the other. Since node placement in the topology can affect the base latency and observed bandwidth, we measure the speedup by taking a ratio of the latency of baseline and *REACT* for the respective placement. We measure the speedup for both the mean and p95 collective completion time over the epochs.

We introduce external flows for congestion in the cluster, from the CPU of an external node to the CPU of a rank running the training job. This external flow is an RDMA ping pong flow that periodically sends 1GB of data. We vary the number of external flows as $p=0,1,2$, to evaluate performance under no congestion, and different degrees of congestion. Table 2 shows a 13-38% speedup offered by *REACT* under congestion ($p=1,2$).

We see a $< 4\%$ drop in performance for when we run the benchmark under no congestion scenario, compared to the baseline. This cost is due to two reasons—overhead of exploration (when we see variable flow performance at runtime) and cost of setting up new connections—without external congestion to be alleviated and compensate for the cost. We try to minimize this cost in our design and discuss in §8 on how to reduce this cost even further. *REACT* is thus a feasible design that improves performance of communication tasks for distributed training workloads in a shared GPU cluster, with low overheads.

7.3 Simulations

We now evaluate *REACT* under a variety of congestion scenarios (§7.1) and under different network stacks. We use ns3 [49] packet level simulator to simulate a collective communication job. We use DCTCP [3] as the congestion control with Random early detection (RED) queues on switches to mark ECNs (queue thresholds set as 32 and 60 for low latency). We size the switch buffers at 32MB and prioritize ACKs in switch queues to prevent delays. For all experiments, we use ECMP as the underlying routing scheme. Following our discussion with the industry, we change ports every epoch to allow ECMP to not be stuck with a bad path choice. Since creating new connections is costly for each epoch, each flow in the collective pattern is initialized with 4 connections on 4 different ports respectively. At each epoch, one port is chosen at random to send the data. We simulate with all links set to 100Gbps and link latency as $1\mu s$. Unless specified, for all experiments we simulate an 8-node training job for 100 epochs with a message size of 20MB. We only measure the mean algorithm bandwidth of a job after 25 epochs to allow initial epochs for *REACT* to converge.

7.3.1 Congestion due to varying number of external flows

We simulate different collectives (Tree AllReduce and Recursive Doubling AllGather) under changing number and degrees of external congested flows on a 3-layer Clos topology. We find that the performance deterioration is different when different ranks in a collective pattern are subjected to congestion from external

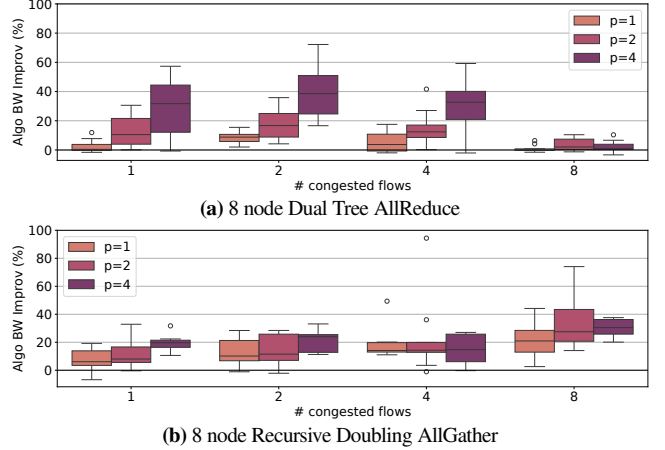


Figure 5: Mean algorithm bandwidth improvement for a sweep of 10 randomly generated scenarios under various degrees of network congestion on a Fattree topology and 20MB message size. x -axis marks the number of source and destination pairs that initiate p external flows each.

flows. To make a more comprehensive comparison, for each case of number of congested flows (source-destination pairs of external flows, x -axes in Fig 5) and degree of congestion (number of flows between same source-destination pair, p), we randomly select the source (from nodes participating in collective) and destination ranks (remaining nodes in the topology) for background traffic. We randomly generate 10 scenarios for each case and plot the percentage improvement in the mean algorithm bandwidth over the epochs in Fig 5. We find that even in low congestion scenarios, $p=1$, *REACT* can help in alleviating performance by upto 20% for both collectives in the worst case scenario— if the cluster is in a particularly bad configuration. In high congestion scenario ($p=2,4$), we find that *REACT* can alleviate performance upto 60-95%, with mean across scenarios being 35% and 15% for tree and recursive doubling respectively. *Note:* In the case of 8 congested flows, all nodes in the training job for tree AllReduce are congested allowing for no room for swaps. Similarly, Recursive Doubling benefits by reducing the path distance for later time steps (avoiding Spine-ToR congestion) when message sizes are large, and hence it is still able to be improved upon in high congestion scenarios.

7.3.2 Tail Latency and effects of ECMP.

We further evaluate the tail epoch communication times, with and without *REACT* for different message sizes, and the effects of varying the number of ECMP port choices (see Appendix C.3). We observe that exploiting multiple path choices through ECMP can help reduce congestion on network links as the number of ports increases, similar to the scenarios described in [17]. However, even with more port choices, collective communication performance can still degrade in the presence of background flows, and *REACT* consistently improves performance across different routing configurations. Thus, *REACT* also reduces the need to maintain higher number of ECMP ports or RDMA queue pair connections which reduces the overall memory load on the NICs to allow for other tasks. We also compare performance

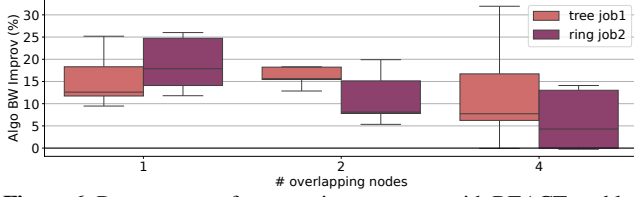


Figure 6: Percentage performance improvement with *REACT* enabled for both jobs compared to without *REACT*, under different degrees of overlap. Job 1 runs 8 node NCCL Tree ALLReduce and Job 2 runs Ring AllReduce. Both run 20MB message sizes on a Star topology. We run 5 random seeds for each overlap scenario.

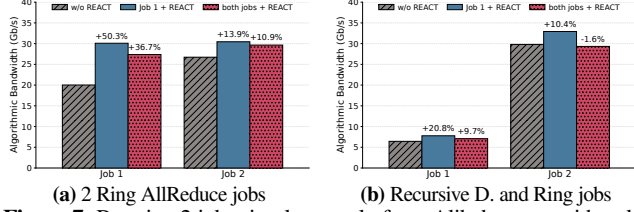


Figure 7: Running 2 jobs simultaneously from Alibaba trace with only network sharing (Spine-ToR congestion). (a) runs two Ring AllReduce jobs simultaneously from Alibaba trace. (b) runs two jobs- one Recursive Doubling algorithm and other with Ring AllReduce- simultaneously. We compare performance when *REACT* is only enabled for Job 1 and when it is enabled for both jobs compared to without *REACT* (gray). The numbers on the bar are the percent change from respective baselines.

of *REACT* for different message sizes in Appendix C.3.

7.3.3 Multiple Jobs.

We now evaluate how *REACT* works when two jobs run simultaneously and affect each other due to node and network sharing. We simulate one job with running Tree AllReduce and another simultaneously running Ring AllReduce on a Star topology. We evaluate with different number of overlapping nodes ie the number of nodes shared by the two jobs (x-axis in Fig 6). We select the overlapped nodes randomly and try with 5 random seeds for each node overlap case. We run both jobs with the original collective pattern and compare it against the scenario where both jobs enable *REACT*. Fig 6 shows the performance improvement for both jobs under different overlapping scenarios.

Fig 7 shows two jobs simulated to run simultaneously on the network topology from Alibaba trace [21] with only network being shared, and no nodes overlap (Spine-ToR congestion). It shows two evaluation scenarios where the two jobs run different communication collectives, thus having different network flow profiles during runtime. We first enable *REACT* only for Job 1 and see a significant improvement in performance for both jobs – enabling *REACT* can help background traffic as well. Even in spine-ToR congestion scenarios, *REACT* is able to provide significant improvement.

Next, we enable *REACT* for both jobs in both scenarios and we observe the performance dips compared to 1 job case – this is due to *REACT* on both jobs interacting independently and getting stuck in bad configuration compared to just 1 job case. Nonetheless, we still see notable improvement in performance due to *REACT*. We leave making *REACT* interact constructively

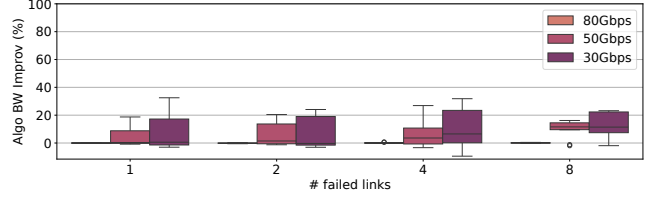


Figure 8: 8 node nccl Tree Allreduce, 20MB message size, Fattree topology, partial link failures.

for all jobs as future work.

7.3.4 REACT under Network Failures

We simulate gray/partial link failures [7, 22] in the 3-layer Clos topology by reducing the link bandwidth of randomly chosen links from anywhere in the three layers to 30Gbps, 50Gbps and 80Gbps. We still use 4-way ECMP so the bandwidth of any flow over the epochs does not directly reduce to the failed link. Fig 8 shows the algorithm bandwidth improvement due to *REACT* for the Tree AllReduce collective– in the worst case, it can help improve performance by upto 20%.

8 Discussion and Limitations

Hierarchical Collectives. AI training jobs may use different hierarchies of collectives to implement 3D parallelism [40, 41] or just divide communication into macro steps. In such cases, we can treat each level of collectives as independent jobs and apply *REACT* to it, treating other collective calls within the larger job as independent smaller AI jobs.

Congestion Scenarios. We explore unknown network paths when we swap nodes in a collective pattern, this will only work if only a limited number of nodes are congested at a time. If most links are congested, no replacement or tuning approach will work to avoid congestion.

Intra-host congestion and collectives. Beyond the scenarios discussed above, *REACT* can also be extended to address issues such as stragglers and host congestion. Although we do not evaluate these cases in this work, if a node is observed to be straggling, it can be moved to later timesteps of the collective to avoid delaying earlier timesteps. *REACT* can also be extended to intra-server collectives to react to observed host congestion, as well as to support other popular collective operations beyond those discussed in this work.

Reducing overheads. Prior works already talk about reducing overheads for RDMA connection establishment (e.g. [53]). We can leverage such works along with modifying NCCL to implement *REACT* with lower overheads.

9 Conclusion

In this work, we present *REACT*, a system that dynamically adapts communication collective patterns at runtime to mitigate network congestion for AI training workloads. It changes the sources and destinations of flows through transforms of swapping nodes in the collective graph, while ensuring that the initial data exchange operation remains semantically correct. *REACT* operates using only end-host observations, avoiding congested links, without requiring network support or global coordination.

We evaluate *REACT* on a shared cluster, demonstrating 13-35% improvements in algorithm bandwidth, and up to 75% gains in simulation across diverse congestion scenarios. More broadly, our results highlight that restructuring application-level dataflow is a practical and complementary alternative to traditional rate control and routing for managing congestion in distributed training.

References

- [1] Open mpi: Open source high performance computing, 1999.
- [2] Mohammad Al-Fares, Alexander Loukissas, and Amin Vahdat. A scalable, commodity data center network architecture. In *Proceedings of the ACM SIGCOMM 2008 Conference on Data Communication*, SIGCOMM '08, page 63–74, New York, NY, USA, 2008. Association for Computing Machinery.
- [3] Mohammad Alizadeh, Albert Greenberg, David A. Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murari Sridharan. Data center tcp (dctcp). In *Proceedings of the ACM SIGCOMM 2010 Conference*, SIGCOMM '10, page 63–74, New York, NY, USA, 2010. Association for Computing Machinery.
- [4] Joao Araujo, Alex Chow, Mark Handley, Ryder Lewis, Christoph Paasch, Jitendra Padhye, Michael Papamichael, Greg Steinbrecher, Amin Tootoonchian, Lihua Yuan, S. Anantharamu, Abhishek Dosi, Mohit Garg, Mahdieh Ghazi, Torsten Hoefler, Deepal Jayasinghe, Jithin Jose, Abdul Kabbani, Guohan Lu, Yang Wang, K. Doddapaneni, Murali Garimella, Vipin Jain, Yanfang Le, H. Nagulapalli, S. Narayanan, Rong Pan, Rathina Sabesan, Raghava Sivaramu, Rip Sohan, Eric Davis, Dragos Dumitrescu, Mohan Kalkunte, Bhaswar Mitra, Guglielmo Morandin, Adrian Popa, Costin Raiciu, Eric Spada, John Spillane, Niranjan Vaidya, Aviv Barnea, Idan Burstein, Elazar Cohen, Yamin Friedman, Noam Katz, Masoud Moshref, Yuval Shpigelman, Shahaf Shuler, Shy Shyman, and Sayantan Sur. Resilient ai supercomputer networking using mrc and srv6, 2026.
- [5] Behnaz Arzani, Siva Kesava Reddy Kakarla, Miguel Castro, Srikanth Kandula, Saeed Maleki, and Luke Marshall. Rethinking machine learning collective communication as a multi-commodity flow problem, 2023.
- [6] Simon Belieres, Mike Hewitt, Nicolas Jozefowicz, and Frédéric Semet. A time-expanded network reduction matheuristic for the logistics service network design problem. *Transportation Research Part E: Logistics and Transportation Review*, 147:102203, 2021.
- [7] Tommaso Bonato, Abdul Kabbani, Ahmad Ghalayini, Michael Papamichael, Mohammad Dohadwala, Lukas Gianinazzi, Mikhail Khalilov, Elias Achermann, Daniele De Sensi, and Torsten Hoefler. Reps: Recycled entropy packet spraying for adaptive load balancing and failure mitigation. In *Proceedings of the 21st European Conference on Computer Systems*, EUROSYS '26, page 225–246, New York, NY, USA, 2026. Association for Computing Machinery.

- [8] Tommaso Bonato, Ales Kubicek, Abdul Kabbani, Ahmad Ghalayini, Maciej Besta, and Torsten Hoefler. Spritz: Path-aware load balancing in low-diameter networks, 2026.
- [9] Zixian Cai, Zhengyang Liu, Saeed Maleki, Madanlal Musuvathi, Todd Mytkowicz, Jacob Nelson, and Olli Saarikivi. Synthesizing optimal collective algorithms. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPOPP '21, page 62–75, New York, NY, USA, 2021. Association for Computing Machinery.
- [10] Jiamin Cao, Yu Guan, Kun Qian, Jiaqi Gao, Wencong Xiao, Jianbo Dong, Binzhang Fu, Dennis Cai, and Ennan Zhai. Crux: Gpu-efficient communication scheduling for deep learning training. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, page 1–15, New York, NY, USA, 2024. Association for Computing Machinery.
- [11] NVIDIA Corporation. Nvidia collective communications library (nccl).
- [12] NVIDIA Corporation. Adaptive routing, 2024.
- [13] NVIDIA Corporation. Networking for the era of ai: The network defines the data center, 2024.
- [14] Facebook. Pytorch: Tensors and dynamic neural networks in python with strong gpu acceleration, 2016.
- [15] Facebook. Gloo: Collective communications library, 2017.
- [16] Clarence Filsfils, Pablo Camarillo, John Leddy, Daniel Voyer, Satoru Matsushima, and Zhenbin Li. Segment Routing over IPv6 (SRv6) Network Programming. RFC 8986, February 2021.
- [17] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. Rdma over ethernet for distributed training at meta scale. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, page 57–70, New York, NY, USA, 2024. Association for Computing Machinery.
- [18] Yanjie Gao, Yichen He, Xinze Li, Bo Zhao, Haoxiang Lin, Yoyo Liang, Jing Zhong, Hongyu Zhang, Jingzhou Wang, Yonghua Zeng, Keli Gui, Jie Tong, and Mao Yang. An empirical study on low gpu utilization of deep learning jobs. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ICSE '24, New York, NY, USA, 2024. Association for Computing Machinery.
- [19] Soudeh Ghorbani, Yimeng Zhao, Srikanth Sundaresan, Ying Zhang, Yijing Zeng, Abhigyan Sharma, Prashanth Kannan, and Cristian Lumezanu. Congestion patterns in a large-scale rdma datacenter. In *Proceedings of the 2025 ACM Internet Measurement Conference*, IMC '25, page 944–951, New York, NY, USA, 2025. Association for Computing Machinery.
- [20] Google. Tensorflow: An end-to-end platform for machine learning, 2015.
- [21] Alibaba Group. Alibaba gpu cluster dataset 2023.
- [22] Vipul Harsh, Tong Meng, Kapil Agrawal, and Philip Brighten Godfrey. Flock: Accurate network fault localization at scale. *Proc. ACM Netw.*, 1(CoNEXT1), July 2023.
- [23] Roger W. Hockney. The communication challenge for mpp: Intel paragon and meiko cs-2. *Parallel Computing*, 20(3):389–398, 1994.
- [24] HPE. Cray slingshot network.
- [25] V. Jacobson. Congestion avoidance and control. *SIGCOMM Comput. Commun. Rev.*, 18(4):314–329, aug 1988.
- [26] Sylvain Jeaugey. Massively scale your deep learning training with nccl 2.4, 2019.
- [27] Myeongjae Jeon, Shivaram Venkataraman, Amar Phanishayee, Junjie Qian, Wencong Xiao, and Fan Yang. Analysis of Large-Scale Multi-Tenant GPU clusters for DNN training workloads. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 947–960, Renton, WA, July 2019. USENIX Association.
- [28] Abdul Kabbani, David J. Wetherall, Gautam Kumar, Junhua Yan, Kira Yin, Masoud Moshref, Mubashir Adnan Qureshi, Qiaobin Fu, Van Jacobson, and Yuchung Cheng. Plb: Congestion signals are simple and effective for network load balancing. 2022.
- [29] Sajy Khashab, Albert Gran Alcoz, Alon Gal, Jacky Romano, Rani Abboud, Yonatan Piasetzky, Lior Maman, Amit Nishry, Barak Gafni, Omer Shabtai, Matty Kadosh, Dror Goldenberg, Gilad Shainer, and Mark Silberstein. High-speed networking for giga-scale ai factories, 2026.
- [30] Ekkehard Köhler, Katharina Langkau, and Martin Skutella. Time-expanded graphs for flow-dependent transit times. In *Proceedings of the 10th Annual European Symposium on Algorithms*, ESA '02, page 599–611, Berlin, Heidelberg, 2002. Springer-Verlag.
- [31] Gautam Kumar, Nandita Dukkupati, Keon Jang, Hassan M. G. Wassel, Xian Wu, Behnam Montazeri, Yaogong Wang, Kevin Springborn, Christopher Alfeld, Michael

- Ryan, David Wetherall, and Amin Vahdat. Swift: Delay is simple and effective for congestion control in the datacenter. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication*, SIGCOMM '20, page 514–528, New York, NY, USA, 2020. Association for Computing Machinery.
- [32] Seunghak Lee, Jin Kim, Xun Zheng, Qirong Ho, Garth A. Gibson, and Eric P. Xing. On model parallelization and scheduling strategies for distributed machine learning. In Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 27. Curran Associates, Inc., 2014.
- [33] Wenxue Li, Xiangzhou Liu, Yunxuan Zhang, Zihao Wang, Wei Gu, Tao Qian, Gaoxiong Zeng, Shoushou Ren, Xinyang Huang, Zhenghang Ren, Bowen Liu, Junxue Zhang, Kai Chen, and Bingyang Liu. Revisiting rdma reliability for lossy fabrics. In *Proceedings of the ACM SIGCOMM 2025 Conference*, SIGCOMM '25, page 85–98, New York, NY, USA, 2025. Association for Computing Machinery.
- [34] Liang Luo, Peter West, Jacob Nelson, Arvind Krishnamurthy, and Luis Ceze. Plink: Discovering and exploiting locality for accelerated distributed training on the public cloud. In I. Dhillon, D. Papailiopoulos, and V. Sze, editors, *Proceedings of Machine Learning and Systems*, volume 2, pages 82–97, 2020.
- [35] Kshiteej Mahajan, Arjun Balasubramanian, Arjun Singhvi, Shivaram Venkataraman, Aditya Akella, Amar Phanishayee, and Shuchi Chawla. Themis: Fair and efficient GPU cluster scheduling. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 289–304, Santa Clara, CA, February 2020. USENIX Association.
- [36] Kshiteej Mahajan, Ching-Hsiang Chu, Srinivas Sridharan, and Aditya Akella. Better together: Jointly optimizing ML collective scheduling and execution planning using SYNDICATE. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 809–824, Boston, MA, April 2023. USENIX Association.
- [37] Sarah McClure, Evyatar Cohen, Alex Shpiner, Mark Silberstein, Sylvia Ratnasamy, Scott Shenker, and Isaac Keslassy. Load balancing for ai training workloads, 2026.
- [38] Behnam Montazeri, Yilong Li, Mohammad Alizadeh, and John Ousterhout. Homa: a receiver-driven low-latency transport protocol using network priorities. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, SIGCOMM '18, page 221–235, New York, NY, USA, 2018. Association for Computing Machinery.
- [39] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. Pipedream: generalized pipeline parallelism for dnn training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, SOSP '19, page 1–15, New York, NY, USA, 2019. Association for Computing Machinery.
- [40] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '21, New York, NY, USA, 2021. Association for Computing Machinery.
- [41] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '21, New York, NY, USA, 2021. Association for Computing Machinery.
- [42] NVIDIA. NCCL Documentation: Collective Operations. <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/collectives.html>, 2026. Accessed: 2026-04-23.
- [43] Pitch Patarasuk and Xin Yuan. Bandwidth optimal all-reduce algorithms for clusters of workstations. *Journal of Parallel and Distributed Computing*, 69(2):117–124, 2009.
- [44] Yanghua Peng, Yibo Zhu, Yangrui Chen, Yixin Bao, Bairen Yi, Chang Lan, Chuan Wu, and Chuanxiong Guo. A generic communication scheduler for distributed dnn training acceleration. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, SOSP '19, page 16–29, New York, NY, USA, 2019. Association for Computing Machinery.
- [45] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, Chao Wang, Peng Wang, Pengcheng Zhang, Xianlong Zeng, Eddie Ruan, Zhiping Yao, Ennan Zhai, and Dennis Cai. Alibaba hpn: A data center network for large language model training. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, page

691–706, New York, NY, USA, 2024. Association for Computing Machinery.

- [46] Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R. Ganger, and Eric P. Xing. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pages 1–18. USENIX Association, July 2021.
- [47] Sudarsanan Rajasekaran, Manya Ghobadi, and Aditya Akella. CASSINI: Network-Aware job scheduling in machine learning clusters. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 1403–1420, Santa Clara, CA, April 2024. USENIX Association.
- [48] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models, 2020.
- [49] George F. Riley and Thomas R. Henderson. *The ns-3 Network Simulator*, pages 15–34. Springer Berlin Heidelberg, Berlin, Heidelberg, 2010.
- [50] Peter Sanders, Jochen Speck, and Jesper Larsson Träff. Two-tree algorithms for full bandwidth broadcast, reduction and scan. *Parallel Comput.*, 35(12):581–594, dec 2009.
- [51] Daniele De Sensi, Tommaso Bonato, David Saam, and Torsten Hoefler. Swing: Short-cutting rings for higher bandwidth allreduce, 2024.
- [52] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. TACCL: Guiding collective algorithm synthesis using communication sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 593–612, Boston, MA, April 2023. USENIX Association.
- [53] Huijun Shen, Jian Yang, Zelong Yue, Xingyu Guo, Xijin Yin, Lang An, Yulin Chen, Jie Ding, Hongyu Wu, Yong Zhang, Jianxi Ye, and Guo Chen. Ucm: Fast and maintainable user-space rdma connection setup. In *Proceedings of the 9th Asia-Pacific Workshop on Networking, APNET '25*, page 24–30, New York, NY, USA, 2025. Association for Computing Machinery.
- [54] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2020.
- [55] Min Si, Pavan Balaji, Yongzhou Chen, Ching-Hsiang Chu, Adi Gangidi, Saif Hasan, Subodh Iyengar, Dan Johnson, Bingzhe Liu, Regina Ren, Deep Shah, Ashmitha Jeevaraj Shetty, Greg Steinbrecher, Yulun Wang, Bruce Wu, Xinfeng Xie, Jingyi Yang, Mingran Yang, Kenny Yu, Minlan Yu, Cen Zhao, Wes Bland, Denis Boyda, Suman Gumudavelli, Prashanth Kannan, Cristian Lumezanu, Rui Miao, Zhe Qu, Venkat Ramesh, Maxim Samoylov, Jan Seidel, Srikanth Sundaresan, Feng Tian, Qiye Tan, Shuqiang Zhang, Yimeng Zhao, Shengbao Zheng, Art Zhu, and Hongyi Zeng. Collective communication for 100k+ gpus, 2026.
- [56] Arjun Singhvi, Nandita Dukkupati, Prashant Chandra, Hassan M. G. Wassel, Naveen Kr. Sharma, Anthony Rebello, Henry Schuh, Praveen Kumar, Behnam Montazeri, Neelesh Bansod, Sarin Thomas, Inho Cho, Hyejeong Lee Seibert, Baijun Wu, Rui Yang, Yuliang Li, Kai Huang, Qianwen Yin, Abhishek Agarwal, Srinivas Vaduvatha, Weihuang Wang, Masoud Moshref, Tao Ji, David Wetherall, and Amin Vahdat. Falcon: A reliable, low latency hardware transport. In *Proceedings of the ACM SIGCOMM 2025 Conference, SIGCOMM '25*, page 248–263, New York, NY, USA, 2025. Association for Computing Machinery.
- [57] Amir Tafreshian, Mojtaba Abdolmaleki, Neda Masoud, and Huizhu Wang. Proactive shuttle dispatching in large-scale dynamic dial-a-ride systems. *Transportation Research Part B Methodological*, 150:227–259, 06 2021.
- [58] Rajeev Thakur, Rolf Rabenseifner, and William Gropp. Optimization of collective communication operations in mpich. *The International Journal of High Performance Computing Applications*, 19(1):49–66, 2005.
- [59] Hao Wang, Han Tian, Jingrong Chen, Xinchun Wan, Jiacheng Xia, Gaoxiong Zeng, Wei Bai, Junchen Jiang, Yong Wang, and Kai Chen. Towards Domain-Specific network transport for distributed DNN training. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 1421–1443, Santa Clara, CA, April 2024. USENIX Association.
- [60] Jiali Wang, Yankui Wang, Mingcong Han, and Rong Chen. Colocating ml inference and training with fast gpu memory handover. In *Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference, USENIX ATC '25*, USA, 2025. USENIX Association.
- [61] Ertza Warraich, Omer Shabtai, Khalid Manaa, Shay Vargaftik, Yonatan Piasetzky, Matty Kadosh, Lalith Suresh, and Muhammad Shahbaz. OptiReduce: Resilient and Tail-Optimal AllReduce for distributed deep learning in the cloud. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 685–703, Philadelphia, PA, April 2025. USENIX Association.
- [62] William Won, Midhilesh Elavazhagan, Sudarshan Srinivasan, Swati Gupta, and Tushar Krishna. Tacos:

Topology-aware collective algorithm synthesizer for distributed machine learning. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, page 856–870. IEEE, November 2024.

- [63] Guanbin Xu, Zhihao Le, Yinhe Chen, Zhiqi Lin, Zewen Jin, Youshan Miao, and Cheng Li. AutoCCL: Automated collective communication tuning for accelerating distributed and parallel DNN training. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 667–683, Philadelphia, PA, April 2025. USENIX Association.
- [64] Hao Zhang, Zeyu Zheng, Shizhen Xu, Wei Dai, Qirong Ho, Xiaodan Liang, Zhiting Hu, Jinliang Wei, Pengtao Xie, and Eric P. Xing. Poseidon: An efficient communication architecture for distributed deep learning on GPU clusters. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*, pages 181–193, Santa Clara, CA, July 2017. USENIX Association.
- [65] Xiaoyang Zhao, Zhe Zhang, and Chuan Wu. Adapcc: Making collective communication in distributed machine learning adaptive. In *2024 IEEE 44th International Conference on Distributed Computing Systems (ICDCS)*, pages 25–35, 2024.
- [66] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. Congestion control for large-scale rdma deployments. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication, SIGCOMM '15*, page 523–536, New York, NY, USA, 2015. Association for Computing Machinery.

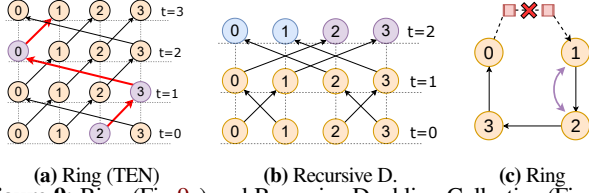


Figure 9: Ring (Fig 9a) and Recursive Doubling Collective (Fig 9b) patterns. Fig 9c shows a Global permutation Swap for Ring AllReduce. If $0 \rightarrow 1$ is congested at the link between two red switches, swapping 1 and 2 can alleviate congestion.

A More Details on Collective and Transforms

A.1 Communication Collective patterns

We briefly explain below the collectives we target with *REACT*: **Ring AllReduce:** A *Ring* based AllReduce [43, 58] connects all N nodes in a ring and divides the message data M into N equal sized chunks. At each timestep, each node sends a chunk to its left neighbor and receives one from its right neighbor. In $N - 1$ steps, all nodes have reduced one chunk of data, hereby completing a reduce-scatter. In next $N - 1$ steps, the ring completes an AllGather for the reduced chunks. The *Ring AllGather* is also implemented in a similar way but the complete message is sent in each timestep and it only requires $N - 1$ steps.

Tree AllReduce: A *Tree* based AllReduce [26, 50] arranges the nodes in a binary tree format, Fig 1b, and data is exchanged along the edges. At the end of the initial exchange, all data is reduced at the root of the tree. Next, the data is broadcast along the reverse tree following down from the root. This algorithm is further optimized by dividing the message M into two chunks and constructing two logical trees, as shown in Fig 1. This is the dual binary tree as implemented by NCCL. The benefit of using a ring is that it uses $O(N)$ steps but is bandwidth optimal as all nodes continuously send data. Tree based AllReduce has $O(\log N)$ steps and scales better [26].

Recursive Doubling AllGather: A Recursive doubling AllGather [58] executes $\log N$ steps, in each timestep s , a node with rank r sends data to $q = r \text{ XOR } 2^s$. In each time step, the size of data sent is doubled. Generally, recursive doubling only works if N is a power of 2 but it can be adapted to work for other values of N .

A.2 Applying Transforms on Collectives

We apply and evaluate the transforms from §5.3 for Ring AllReduce and Recursive Doubling AllGather below. Table 1 summarizes the transforms and types of congestion each transform is able to alleviate.

Tree AllReduce: As discussed in §5.3, all three transforms are able to alleviate the various congestion scenarios.

Ring AllReduce: Fig 9a shows that when the $0 \rightarrow 1$ link experiences Spine-ToR congestion, swapping all instances of node 1 with node 2 can help alleviate it (as per the global swap transform ①). Since all nodes in the ring have one incoming and one outgoing flow, if $0 \rightarrow 1$ flow is Server-TOR congested, then no swaps are helpful as the in and out-degrees remain the same.

The red edges in Fig 9a highlight one of the chunk-graphs present, with violet nodes highlighting equivalent swappable nodes. Swapping nodes within the chunk graph (②, ③) is not helpful in the Ring AllReduce as nodes in the final collective pattern will end up with a higher degree (swapping 0 (violet) with 3 (violet) will lead to higher degree for nodes 0 and 3 at time steps 1 and 2 respectively).

Recursive Doubling AllGather incurs larger message sizes in later timesteps. Global Swap Transform ① helps alleviate congestion by assigning lower-cost flows to later timesteps, at the expense of assigning higher-cost flows to earlier timesteps when message sizes are smaller. Since there is only one chunk graph, Chunk-Swap Transform ② is already addressed by ①. The blue and purple nodes in Fig. 9b represent two equivalent swappable sets (③) for collective pattern, which can help mitigate spine-ToR congestion scenarios (i.e., erratic congestion).

A.3 Analyzing Position-equivalent Swaps

We briefly explain the brute force algorithm used to find all Position-equivalent swaps for a given TEN graph and its complexity.

We assign all nodes a one hot-encoded bitvector with a single bit set to 1 based on rank. We simulate an execution of the collective operation on this vector by running a breadth-first traversal of the collective graph and checking whether the final bitvector on all nodes aligns with the conditions of the collective operation. For a graph $G(V, E)$ with vertices V and edges E , this traversal will take $O(|V| + |E|)$ time. If after the traversal, all nodes have all bits (or corresponding bits for the collective) as 1, we know the collective is semantically correct. For all pairwise vertices in the TEN, we swap them and conduct this brute force safety check. This gives a complexity of $O(|V|^2(|V| + |E|))$. For a general collective pattern with N ranks and T steps, $|V| = N \cdot T$, and for general collectives $T = O(N)$. Hence the complexity could reach up to $O(N^7)$. But for the popular and high performing collectives that are pareto-optimal [9], we observe TEN has $|V| + |E| = O(N \log N)$. Hence, for popular collectives we have the complexity as $O((N \log N)^3)$.

A.4 Summarizing REACT Parameters

Table 3 gives a summary of all parameters used by *REACT*.

Parameters	Definition
B	Batch size of epochs
δ_1	Steady congestion parameter (§5.2)
δ_2	irregular congestion parameter (§5.2)
K_t	Number of trials each turn
ϵ	Threshold of min. expected perf. improvement
H	How much older data we maintain
E_{max}	Number of exploratory rounds
A_{max}	Maximum exploration attempts
τ_r	Revert threshold to measure perf. degradation

Table 3: Parameters used by *REACT*

A.5 Feedback Loop Algorithm

Algorithm 1 gives the algorithm we use in §5.4.1 to apply the swapping transforms to collective patterns.

Input: $G: \{\langle V_c, E_c \rangle | c \in C\}$ TEN graph of algorithm
Input: FCT FCTs of all flows
Input: $\mathcal{D}$ Old FCTs of all flows
Input: r local rank of device
Function ApplyTransforms():
 $P \leftarrow \text{GetCriticalPaths}(G, \text{FCT});$
 $C \leftarrow \text{DetectCongestion}(\text{FCT});$
 $X \leftarrow C \cap P$
 $X \leftarrow \text{SortCongestedNodes}(X)$
 $u, c \leftarrow 1, 1$
foreach $x \leftarrow \text{pop}(X)$ & $u++ < K_t$ & $c < K_{max}$ **do**
 $V \leftarrow \text{ViableSet}(x, G)$
 $r \leftarrow \text{Random}(V)$
while True and $u++ < K_t$ **do**
 $V \leftarrow V - \{r\}$
 $G' \leftarrow \text{Swap}(G, x, r)$
// compare expected performance
// of G' vs G
if $E_{\mathcal{D}}(G') > (1 + \epsilon) E_{\mathcal{D}}(G)$ **then**
 $G \leftarrow G'$
 $X \leftarrow \text{GetCriticalPaths}(G', \mathcal{D}) \cap C$
 $c \leftarrow c + 1$
break
end
 $r \leftarrow \text{Random}(V)$
end
end
return G ;
End Function

Algorithm 1: Algorithm to execute swapping transforms.

B Implementation Details

Fig 10 shows the intra-node topology and the Cray Slingshot [24] network topology as deployed in the shared cluster. We run our experiments with GPU Direct RDMA enabled. The blue nodes in Fig 10b show an example ML job running on 4 nodes and the red line is an injected congestion flow, as we inject in §7.2.

Adaptive Routing. Note that we also enable adaptive routing in our deployment. The red flow is preferably routed along the shortest path as shown. In case $\text{TOR}_1 \rightarrow \text{TOR}_0$ link is congested, the switch can autonomously decide to load balance the red flow to route via TOR_2 . Note that *REACT* does not modify/control this routing decision but works regardless of this load balancing scheme.

B.1 Implementation of CTCollective

Algorithm 2 shows the pseudo code we use to implement the

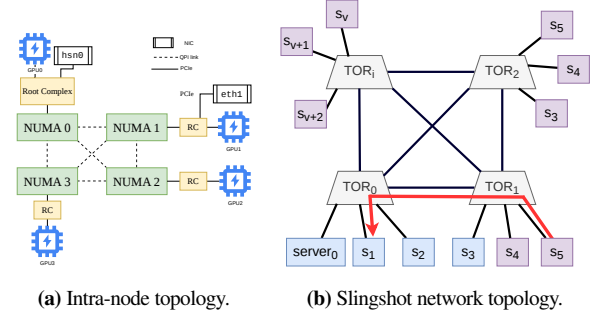
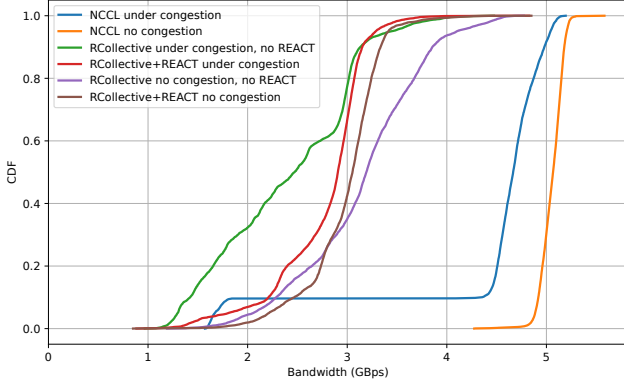


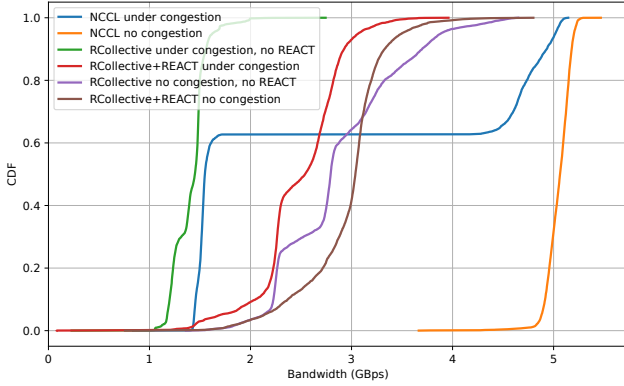
Figure 10: The intra-node and network topology in the shared academic GPU cluster. Within a server, four A100 GPUs are interconnected by NVLink (not shown for clarity). The ToRs in Slingshot network topology are all fully connected with each other. The hsn-high speed interface on each server is connected to exactly one TOR. Blue nodes show an example 4 node job, red arrows marks an external congested flow injected to model congestion.

Input: $G: \{\langle V_c, E_c \rangle | c \in C\}$ TEN graph of algorithm
Input: r local rank of device
Function RunCollective(D):
 $m \leftarrow \text{MaxDegree}(r, G);$
 $S \leftarrow \text{CreateStreams}(|C| \times m);$
 $G_r \leftarrow \text{LocalGraph}(r, G)$
foreach $c \in C$ **do**
// for each chunk sub-graph
foreach $t \in G_r$ **do**
foreach $e \in E(c, t, r)$ **do**
 $\text{src}, \text{dst} \leftarrow e$
if $r == \text{src}$ **then**
stream $S[c \cdot m + e]:$
post_to_gpu(isend(dst))
end
else
stream $S[c \cdot m + e]:$
post_to_gpu(irecv(src))
end
end
block_streams($S[c \cdot m : (c+1) \cdot m - 1]$)
if $\text{agg} \in E(c, t, r)$ **then**
stream $S[c \cdot m]: \text{post_to_gpu}(\text{agg})$
end
block_streams($S[c \cdot m : (c+1) \cdot m - 1]$)
end
end
return F ;
End Function

Algorithm 2: Function for implementing any custom graph in CTCollective library



(a) *REACT* under $p=2$ congestion.



(b) *REACT* under $p=2$ congestion.

Figure 11: CDF of epoch times for TreeAllReduce measured on the shared GPU cluster.

C Additional Evaluation Results

C.1 Testbed evaluation

In Fig 11, we plot the CDF of epoch times from one of the runs in §7.2 for 4 nodes. We compare performance of baseline (RCollective with no REACT) and *REACT* (RCollective+REACT) in $p=1,2$ congestion scenarios. We also plot the $p=0$ scenarios as “no congestion” lines in both plots of Fig 11. As we compare the red (*REACT*) and green (baseline) lines in both plots, using *REACT*, we are able to alleviate congestion. We also come very near to the purple plot (baseline under no congestion). The brown (*REACT*) line is close to the baseline (purple) under no congestion.

Comparison with NCCL. Finally, we plot the NCCL with and without external congestion with blue and orange respectively. External congestion affects NCCL’s performance in both cases. RCollective performs worse than NCCL in most cases due to the overheads of p2p communicators. Note that under $p=2$, RCollective+REACT performs better than NCCL under congestion for a majority of the epochs.

C.2 Results on Star Topology

We also evaluate experiments with external congested flows for the Tree Allreduce by ns-3 simulations on the Star topology in Fig 12. We find that higher improvements in this case than in Fattree (Fig 5a)– due to fixed paths in star topology, *REACT* is able to make better decisions.

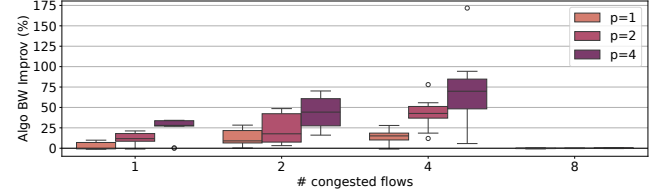
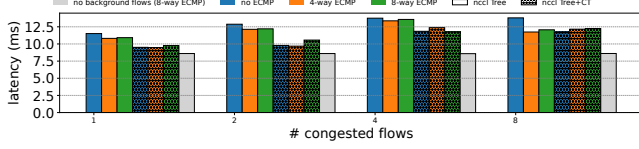


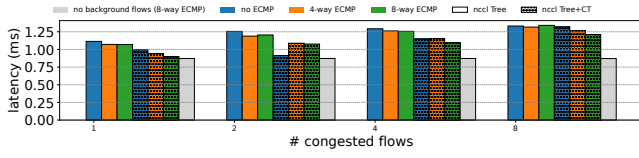
Figure 12: 8 node nccl Tree Allreduce, 20MB message size, Star topology

C.3 Tail Latency and effects of ECMP.

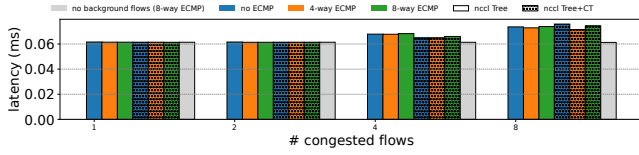
Fig 13 shows the p95 epoch time across all randomly generated scenarios for number and degree of congested flows in Fig 5a. We run the same scenario (same network paths and random seed) under no congestion, as well as under congestion with different number of ECMP port choices, 1 and 8 respectively. We find that exploiting the multiple path choices through ECMP helps amortize any congested links in the network as we increase the number of port. Still, performance degrades of collective communication degrades and *REACT* helps improve performance regardless of the underlying routing configuration/solution. We find that, with our short queue thresholds at the switches, collectives with smaller message sizes ($\leq 100\text{KB}$) are not significantly affected by congested flows at lower p (Fig 13c). At higher p , Fig 13e, our solution still improves performance for these message sizes.



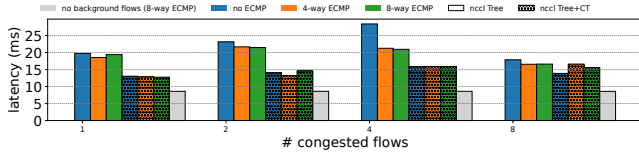
(a) 8 node nccl Tree Allreduce, 20MB message size, congestion degree $p=1$



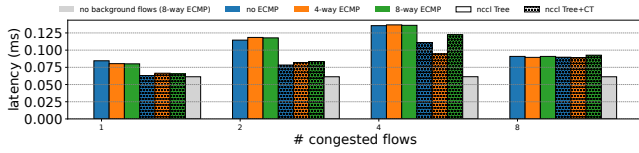
(b) 8 node nccl Tree Allreduce, 2MB message size, congestion degree $p=1$



(c) 8 node nccl Tree Allreduce, 100KB message size, congestion degree $p=1$



(d) 8 node nccl Tree Allreduce, 20MB message size, congestion degree $p=4$



(e) 8 node nccl Tree Allreduce, 100KB message size, congestion degree $p=4$

Figure 13: Algorithm bandwidth improvement in a fattree topology for the p95 case. Comparing the effects of ECMP and collective tuning in alleviating latency deterioration due to network congestion. Increasing ECMP ports/QPs comes at additional cost while collective tuning works even with less QP configurations.